一、数据类型

1.1 类型

■ 计算机中的数据以二进制的形式存储在寄存器或存储器中。

■ 汇编语言中的数据类型取决于指令操作码。

- ☐ 存储在寄存器、存储器中的操作数本身没有数据类型，对该数进行何种数据类型的操作完全取决于指令。
- ☐ 同一个操作数，既可以当作有符号数，也可以当作无符号数；既可以是定点数，也可以是浮点数。

■ 高级语言具有数据类型，下面以C语言为例进行介绍。



注：寄存器中的操作数本身不存其是何种操作类型，对该数进行何种操作，取决于指令

1.2 整型的取值范围

C 语言中整型变量的取值范围

		char（8 位）	short（16 位）	int（32位）	long（64 位）
无符号 (unsigned) 最大值	机器码	0xFF	0xFFFF	0xFFFF FFFF	0xFFFF FFFF FFFF FFFF
	真值	255	65,535	4,294,967,295	18,446,744,073,709,551,615
有符号 最大值	机器码	0x7F	0x7FFF	0x7FFF FFFF	0x7FFF FFFF FFFF FFFF
	真值	127	32,767	2,147,483,647	9,223,372,036,854,775,807
有符号 最小值	机器码	0x80	0x8000	0x8000 0000	0x8000 0000 0000 0000
	真值	−128	−32,768	−2,147,483,648	−9,223,372,036,854,775,808
−1	机器码	0xFF	0xFFFF	0xFFFF FFFF	0xFFFF FFFF FFFF FFFF
0	机器码	0x00	0x0000	0x0000 0000	0x0000 0000 0000 0000

■ 无论是无符号数还是有符号数，C语言程序并不检测数据在加、减、乘等运算中产生的溢出现象。

- ☐ 程序员应尽量避免出现这种情况，所编制的应用程序应具有对溢出进行判断的功能。

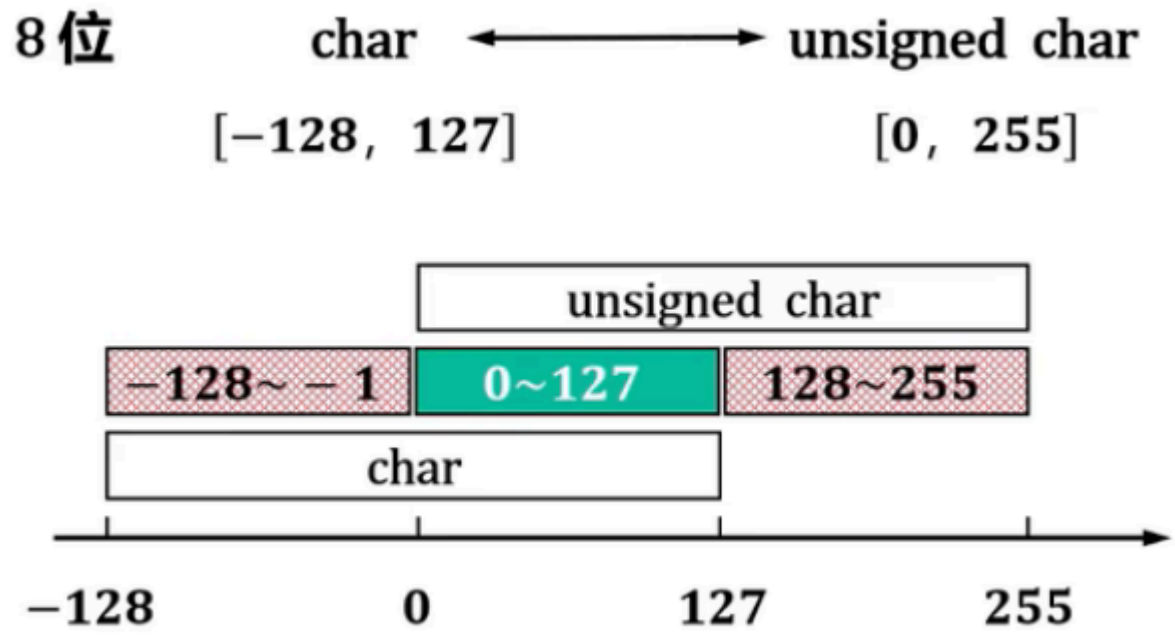
注：C语言不自动检测溢出，参考[[C语言不检测数据溢出的特性]]

二、类型转换

2.1 整型之间的转换

2.1.1 相同字长之间的转换

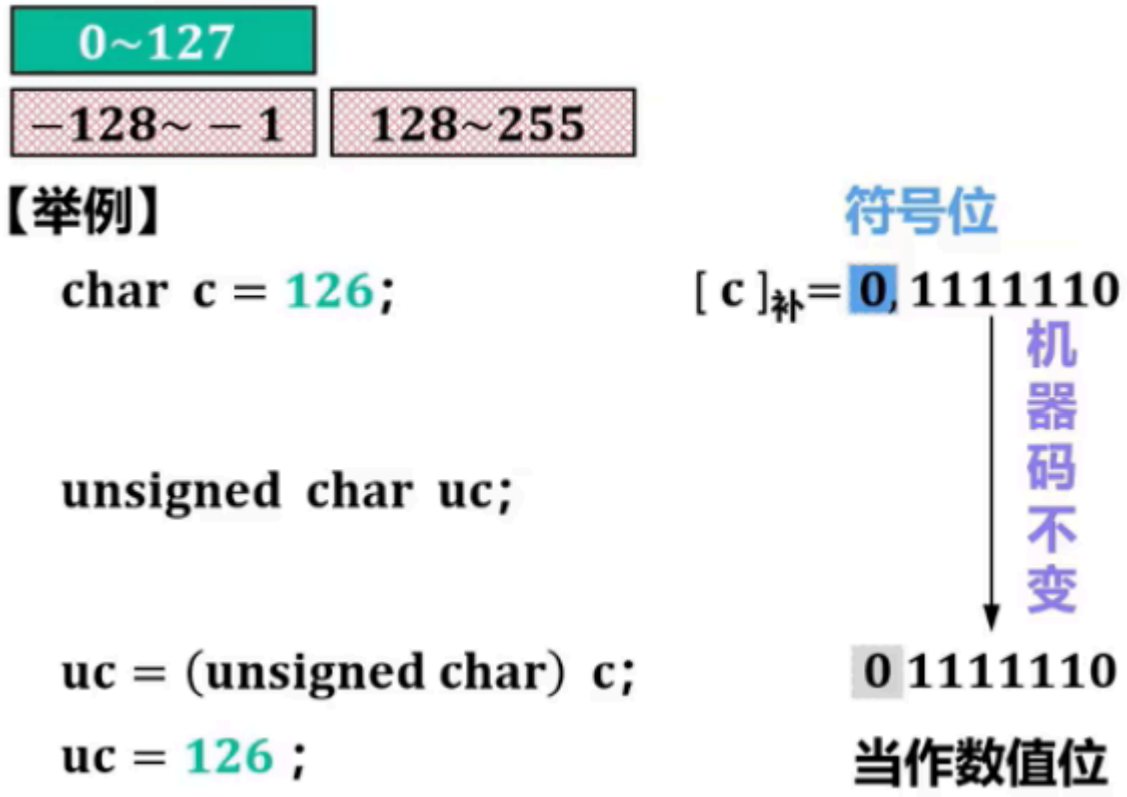
以char与unsigned char举例：



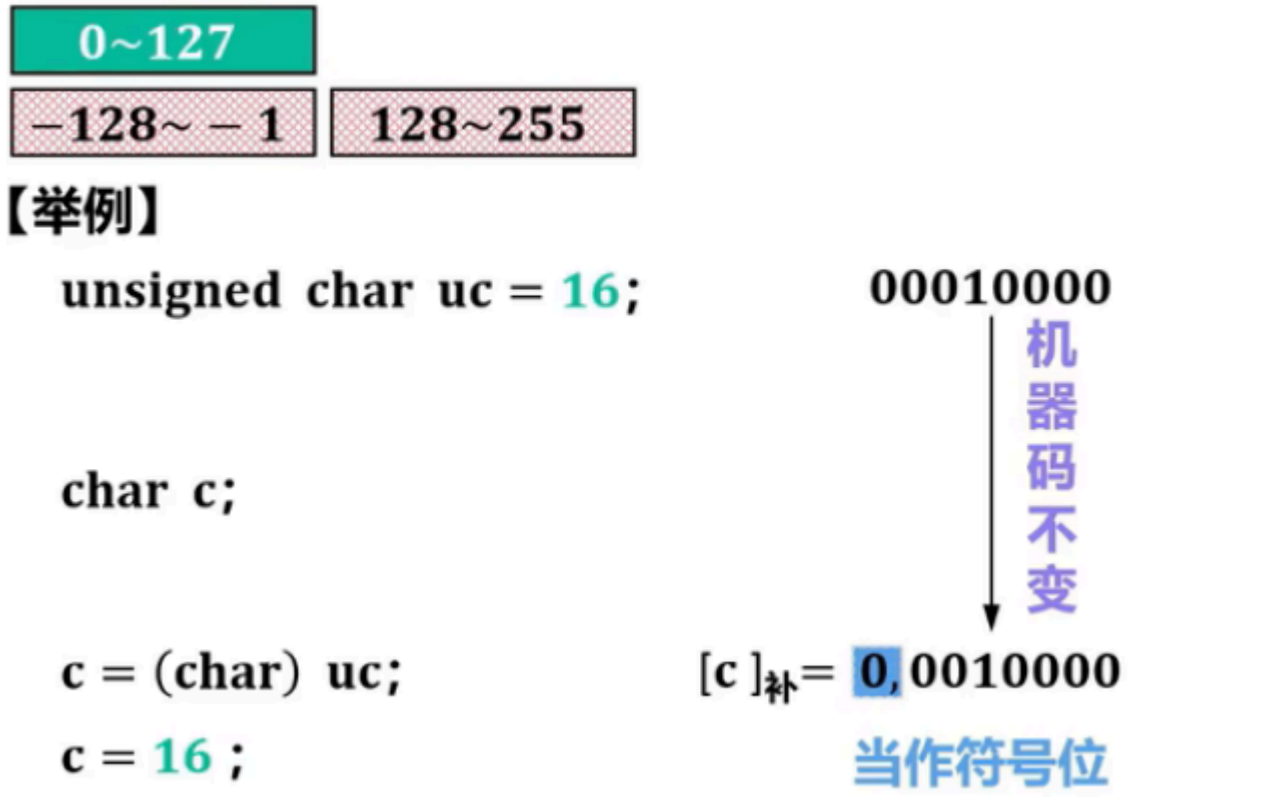
分三种情况：

1. 重叠区域的转换：

char 转 unsigned char



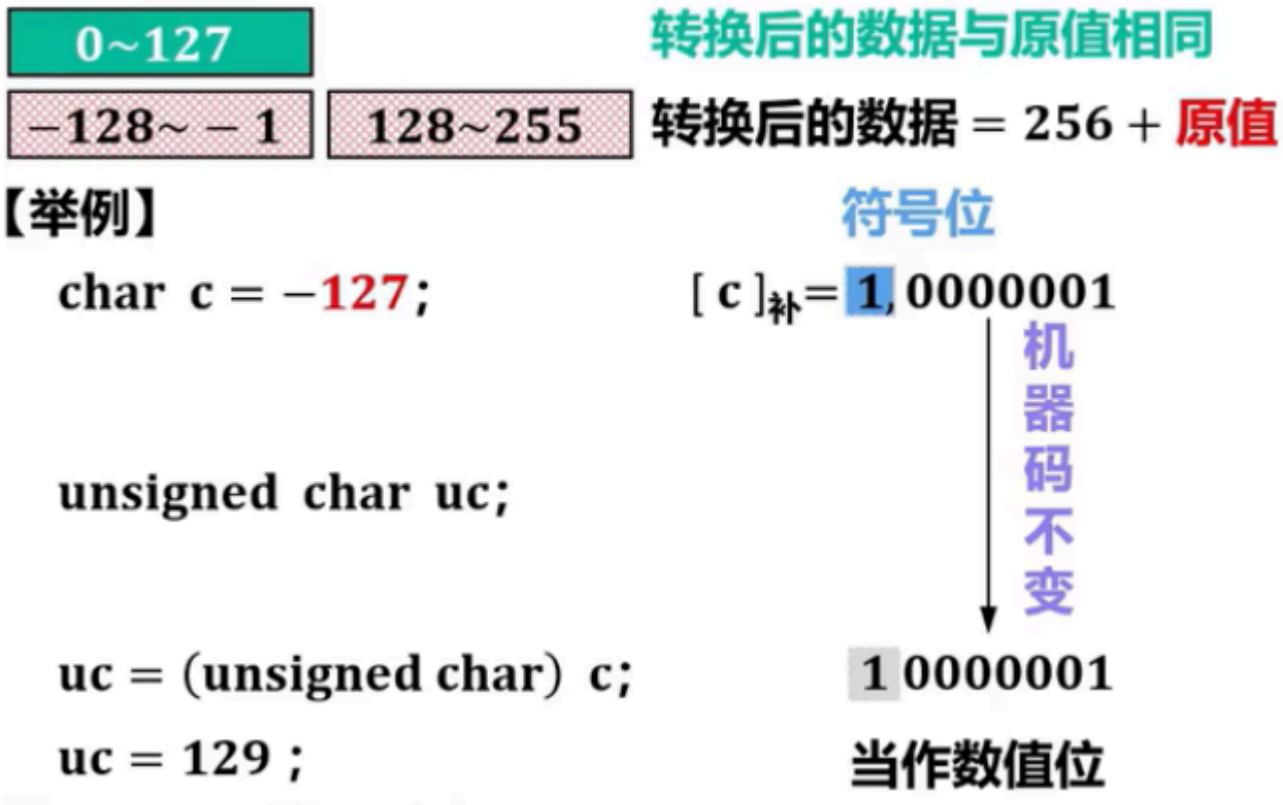
unsigned char 转 char



注：重叠区域为0到127时，最高非0位为第七位，当把第八位视为符号位或者把第八位当作数值位时，不影响真值（因为第八位始终为0），所以重叠区域的转换不改变真值

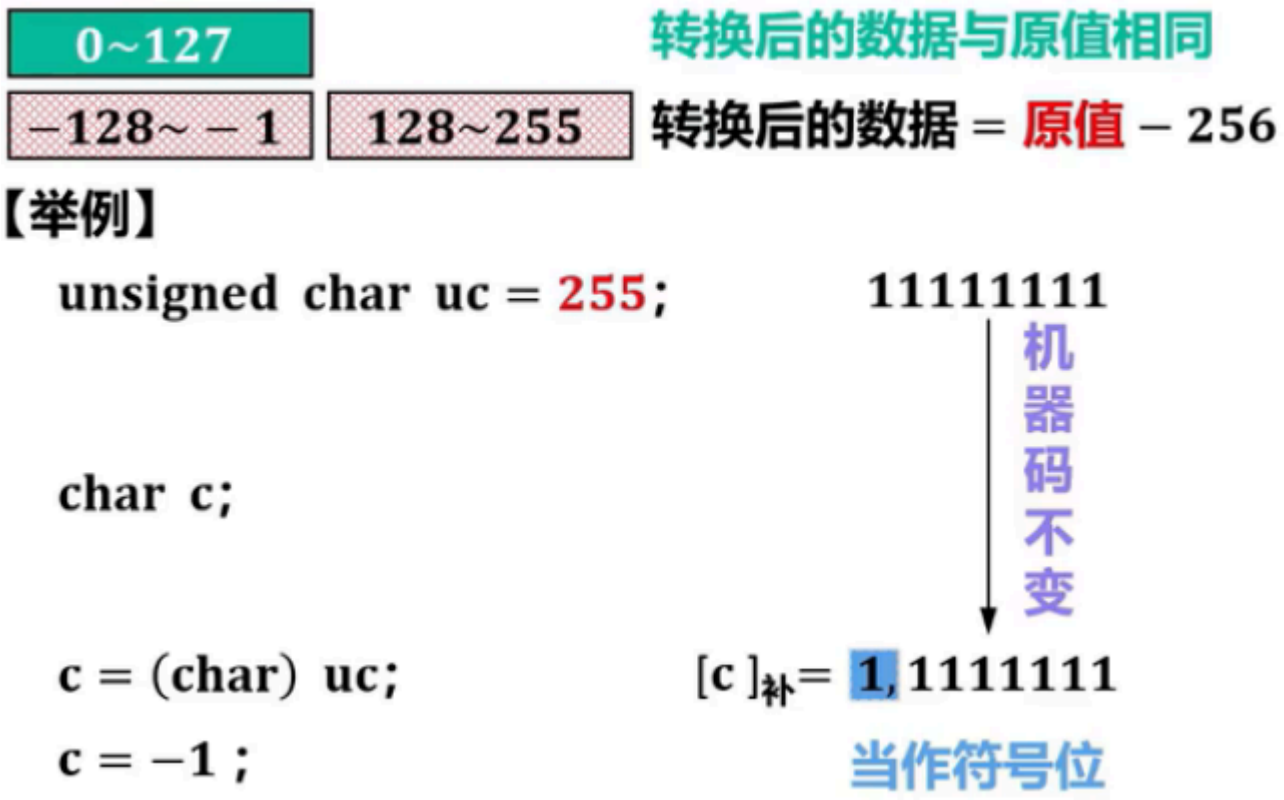
2. 独属char区域

   char 转 unsigned char



注：有符号转无符号，将最高位视为数值为，独属char区域的数均为负数，符号位为1，视为数值位后，做出的贡献为256

3. 独属unsigned char区域

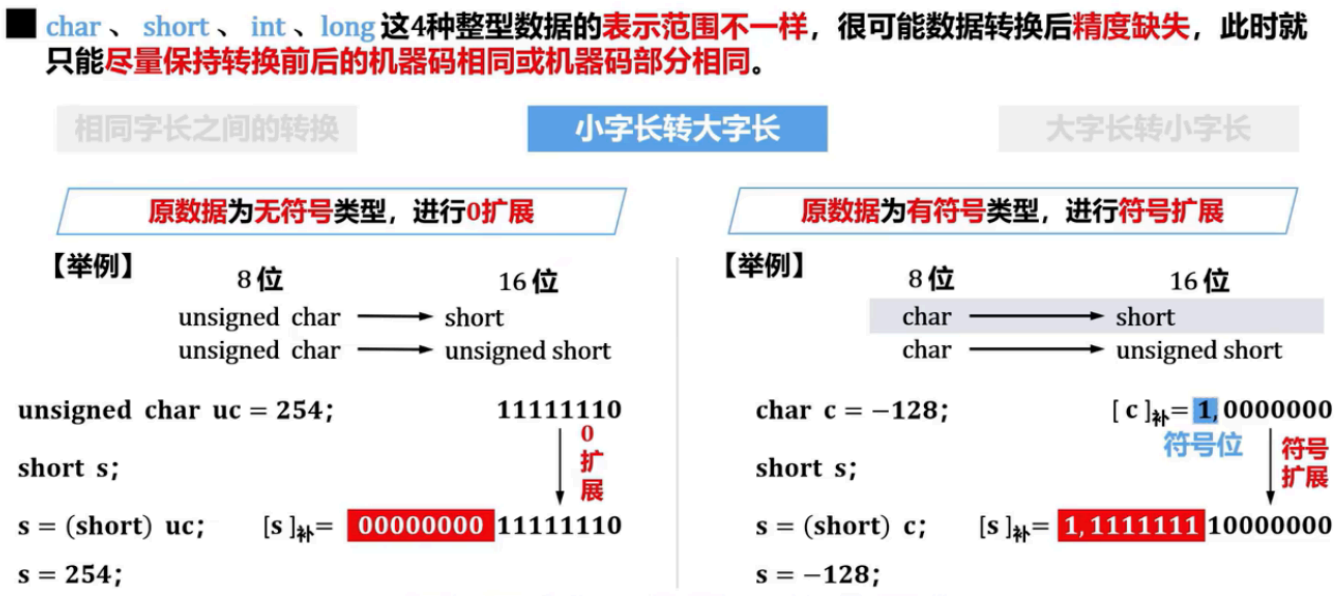


注：无符号转有符号，将最高位视为符号位，独属于unsigned char 区域的数，最高位均为1，视为符号位后，相当于减去256

2.1.2 小字长转大字长

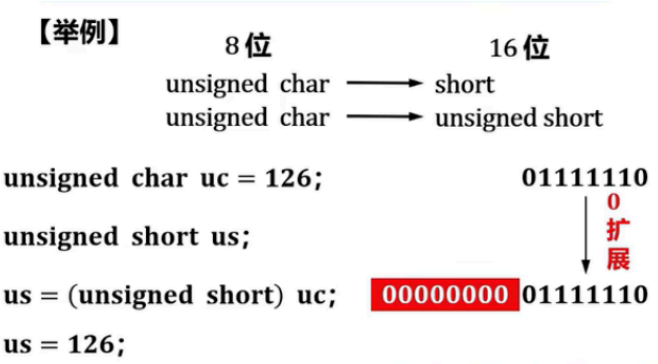
unsigned char 转 short

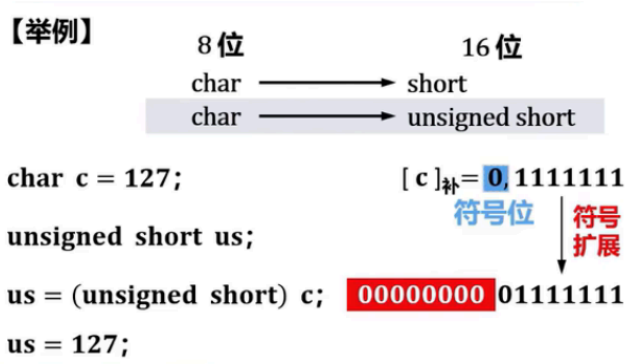
char 转short



unsigned char 转 unsigned short

char 转 unsigned short





注：小字长转大字长的核心在于扩展：

- 原数据为无符号，则0扩展，不会改变真值
- 原数据为有符号，则符号位扩展，对真值的影响分情况：

1. 若符号位为0，即为正数，则不改变真值
2. 若符号位为1，即为负数，则改变真值

2.1.3 大字长转小字长

相同字长之间的转换　小字长转大字长　**大字长转小字长**

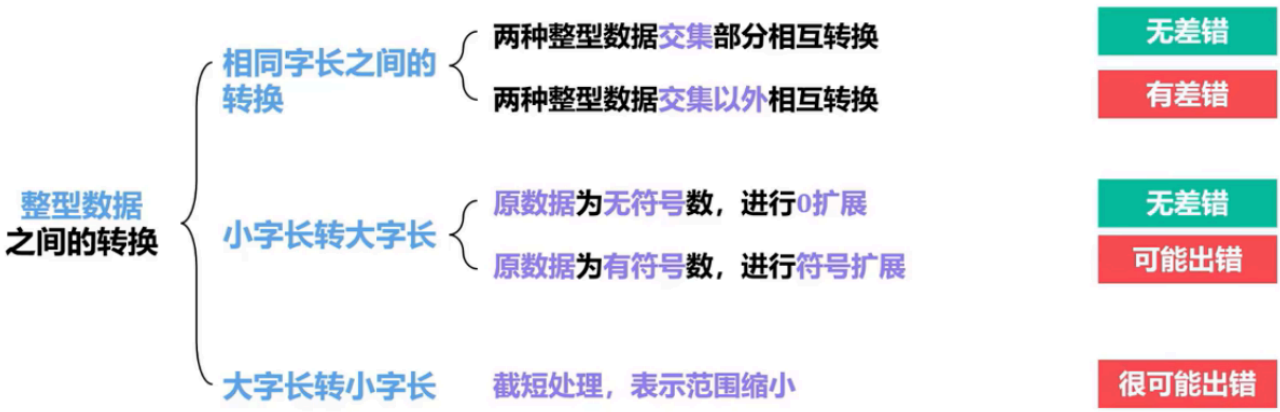
一般情况下：

- 编译器会将机器码截短处理
  - 表示范围缩小
  - 很可能出错

【举例】

```
short s;   unsigned short us;
char  c;   unsigned char uc;
int i = -40832;          [i]补= 1,111 1111 1111 1111 0110 0000 1000 0000   i = -40832
s = (short) i;           [s]补= [redacted]0110 0000 1000 0000              s = 24704
us = (unsigned short) i;        [redacted]0110 0000 1000 0000              us = 24704
c = (char) i;            [c]补= [redacted]1000 0000                        c = -128
uc = (unsigned char) i;         [redacted]1000 0000                        uc = 128
```

2.1.4 整型之间转换的总结



2.2 int、double、float之间的转换

■ int、float、double之间也可以进行强制类型转换。

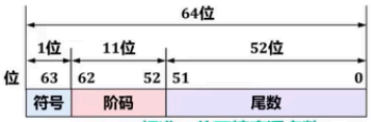
- ☐ 上述3种类型数据的机器码并不相同（int型数据是32位有符号整数，用补码表示；float和double型数据分别是32位和64位浮点数，它们的阶码用移码表示、尾数用原码表示）。
- ☐ 上述3种类型数据的表示范围和精度也不相同。
- ☐ 因此在转换过程中编译器只能保证数值尽量相等，大多数情况下只是近似值。
- ☐ 下面，我们讨论以下几种转换情况。
  - ◇ float → double
  - ◇ double → float
  - ◇ float/double → int
  - ◇ int → float
  - ◇ int → double

2.2.1 float转double

float → double　double → float　float/double → int　int → float　int → double



IEEE 754标准32位单精度浮点数
(对应C语言中的float型)



IEEE 754标准64位双精度浮点数
(对应C语言中的double型)

- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示，偏置常数单精度为127，双精度为1023。
- 尾数：定点小数，用原码表示。符号位前移到最左侧。相邻左侧隐藏一个1，表示数值而不表示符号。对于单精度，尾数实际有24位；对于双精度，尾数实际有53位，但不保存隐藏的那个1，节省的比特位可用于提高尾数的精度。

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

由于double型数据的阶码和尾数的位数都比float型大，因此其表示范围更大、精度更高，转换后的double型数据与原float型数据的值完全相等。

2.2.2 double转float

float → double　　double → float　　float/double → int　　int → float　　int → double

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})$ $\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})$ $\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))$ $\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))$ $\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

- 大数转换：可能发生溢出。例如：`double d = 1234567890123456; float f = (float) d;`
- 高精度数转换：发生舍入。例如：`double d = 1.123456789; float f = (float) d;`

2.2.3 float/double 转 int

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})$ $\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})$ $\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))$ $\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))$ $\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

→

C语言中的int（32位）		
无符号(unsigned)最大值	机器码	0xFFFF FFFF
	真值	4,294,967,295
有符号最大值	机器码	0x7FFF FFFF
	真值	2,147,483,647
有符号最小值	机器码	0x8000 0000
	真值	−2,147,483,648
−1	机器码	0xFFFF FFFF
0	机器码	0x0000 0000

- 小数部分：向0方向截断
- 大数转换：可能发生溢出

2.2.4 int 转float

float → double　　double → float　　float/double → int　　int → float　　int → double

C语言中的int（32位）		
无符号(unsigned)最大值	机器码	0xFFFF FFFF
	真值	4,294,967,295
有符号最大值	机器码	0x7FFF FFFF
	真值	2,147,483,647
有符号最小值	机器码	0x8000 0000
	真值	−2,147,483,648
−1	机器码	0xFFFF FFFF
0	机器码	0x0000 0000

→

数值的分类			单精度（32位，对应C语言中的float）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})$ $\approx 3.4\times 10^{38}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))$ $\approx -3.4\times 10^{38}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$

- 两种类型都是32位，各自的数据组合（状态）数量相同，但二者在数轴上表示的数据并不完全重叠。
  - 由于float型浮点数的尾数包括隐藏位在内共24位，当int型数据的高8位（24~31位）数据为非0时，无法精确转换成24位浮点数的尾数，此时发生精度溢出。

2.2.5 int转double

float → double　double → float　float/double → int　int → float　**int → double**

C语言中的int（32位）		
无符号(unsigned)最大值	机器码	0xFFFF FFFF
	真值	4,294,967,295
有符号最大值	机器码	0x7FFF FFFF
	真值	2,147,483,647
有符号最小值	机器码	0x8000 0000
	真值	−2,147,483,648
−1	机器码	0xFFFF FFFF
0	机器码	0x0000 0000

→

数值的分类			双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

double型数据的尾数包含隐藏位在内有53位，可以精确表示所有32位整数。

2.2.6 总结

C语言中数据类型转换 —— int、float、double之间的转换

float → double	double → float	float/double → int	int → float	int → double
值相同	大数可能溢出，高精度发生舍入	小数部分向0截断，大数可能溢出	比较大的数无法精确表示	值相同

【实例】假设变量i、f、d的类型分别是int、float和double，请判断下列C语言关系表达式是否恒为真?

`i == (int)(float) i;`	不恒为真	i 转换为float型时就可能出现精度缺失，可不考虑再转回int型
`f == (float)(int) f;`	不恒为真	f 转换为int型时就就可能出现小数部分向0截断，可不考虑再转回float型
`i == (int)(double) i;`	恒为真	i 转换为double型后数值不变，因此再转回int型后数值不变
`f == (float)(double) f;`	恒为真	f 转换为double型后数值不变，因此再转回float型后数值不变
`d == (float) d;`	不恒为真	d 转换为float型时可能出现大数溢出和高精度舍入
`f == -(- f);`	恒为真	浮点数取负仅将其数符（最高位）取反即可。